

Studentu sistema

Generated by Doxygen 1.17.0

1 README	1
1.0.1 V0.4 testavimai	1
1.0.2 Padaryti pokyčiai - Struct pakeistas į Class.	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Class Documentation	9
5.1 Studentas< GradeContainer > Class Template Reference	9
5.1.1 Detailed Description	11
5.1.2 Constructor & Destructor Documentation	11
5.1.2.1 Studentas() [1/3]	11
5.1.2.2 Studentas() [2/3]	11
5.1.2.3 Studentas() [3/3]	11
5.1.2.4 ~Studentas()	12
5.1.3 Member Function Documentation	12
5.1.3.1 egzaminas()	12
5.1.3.2 mediana()	12
5.1.3.3 operator=() [1/2]	12
5.1.3.4 operator=() [2/2]	13
5.1.3.5 pazymiai() [1/2]	13
5.1.3.6 pazymiai() [2/2]	13
5.1.3.7 setEgzaminas()	13
5.1.3.8 setMediana()	13
5.1.3.9 setVidurkis()	14
5.1.3.10 tipas()	14
5.1.3.11 vidurkis()	14
5.1.4 Member Data Documentation	14
5.1.4.1 egzaminas_	14
5.1.4.2 mediana_	14
5.1.4.3 pazymiai_	15
5.1.4.4 vidurkis_	15
5.2 Zmogus Class Reference	15
5.2.1 Detailed Description	16
5.2.2 Constructor & Destructor Documentation	16
5.2.2.1 Zmogus() [1/3]	16
5.2.2.2 Zmogus() [2/3]	16
5.2.2.3 Zmogus() [3/3]	16

5.2.2.4 ~Zmogus()	16
5.2.3 Member Function Documentation	17
5.2.3.1 operator=() [1/2]	17
5.2.3.2 operator=() [2/2]	17
5.2.3.3 pavarde()	17
5.2.3.4 setPavarde()	17
5.2.3.5 setVardas()	18
5.2.3.6 tipas()	18
5.2.3.7 vardas()	18
5.2.4 Member Data Documentation	18
5.2.4.1 pavarde_	18
5.2.4.2 vardas_	18
6 File Documentation	19
6.1 funkcijos.h	19
6.2 library.h	28
6.3 main.cpp	28
6.4 objektinis_patikrinimai.cpp	29
6.5 patikrinimai.h	30
6.6 rule_of_five_test.cpp	30
Index	33

Chapter 1

README

Kaip naudotis programa:

1. make clean - išvalyti kompiliacijos failus.
2. make - komanda sukuria failą "programa" pagal makefile.
3. ./programa - komanda paleidžia pačią programą.

Papildoma: Norint atlikti programos testavimą konsolėje reikia įrašyti komandą - make test.

1.0.1 V0.4 testavimai

4 versijoj duomenų įvedimo sistemoj buvo pridėta nauja failų generavimo funkcija, kuri kuria failus pagal šablonus naudotis 2 versijoj. Buvo atlikti testavimai laiko apskaičiavimui ir jų rezultatai bus pateikti žemiau: 1 testavimas: Duoti testavimai turi skirtingus studentų pažymių kiekius, kad matytusi skirtumas tarp atlikimo laikų.

2 testavimas (visi testai buvo atlikti su pažymių sk. = 2 ir rušiuoti pagal medianą):

V1.0 testavimai Sistema: CPU - Apple M2 (8 branduoliai) RAM - 16gb SSD - 256gb

1 strategija - Duomenys nuskaityti į vieną duomenų konteinerį ir skirstymo metu masyvas skirstomas į maladiec ir lopų konteinerius. Rezultatai:

2 strategija - Duomenys nuskaityti į "maladiec" konteinerį ir žmones su mažesniu vidurkiu yra įdedami į lopai konteineri ir iškerpami. Rezultatai:

3 strategija - Padaryta pagal 2 strategiją, tik vietoj erase yra naudojamas partition algoritmas. Rezultatai:

V1.1 TESTAVIMAI

1.0.2 Padaryti pokyčiai - Struct pakeistas į Class.

Greičio patikrinimas po pokyčių: Aiškių skirtumų tarp atlikimo laiko nėra.

Optimizacijos Flagai. Testavimui buvo naudojami optimizacijos flagai -O1, -O2, -O3 ir testavimai buvo atlikti su tais pačiais failais.

-O1:

-O2:

-O3:

Išvados: Matosi aiškus skirtumas tarp programų atlikimo laiko kur nebuvo naudojami optimizacijos flagai ir kur buvo, kadangi skirtumas atlikimo laike vos ne dvigubai skiriasi, bet atlikimo greičio skirtumas tarp pačių flagų nėra aiškiai pastebimas.

V1.2 TESTAVIMAI Padaryti pokyčiai - įgyvendinta "Rule of five" ir įvesties/išvesties operatoriai. Taip pat pridėtas testavimo metodas, kuris patikrina, kad visi metodai veikia.

Apie išvesties ir išvesties metodus:

1. perator>> nuskaityto vardą, pavardę, egzamino pažymį, pažymių kiekį ir pačius pažymius
2. operator<< išveda studento duomenis tuo pačiu formatu į failą arba į konsolę pagal vartotojo pasirinkimą.
3. šie operatoriai buvo testuojami [rule_of_five_test.cpp](#)

Kaip veikia testavimas: Testavimo kodas yra paleidžiamas konsolėje įvedus ./rule_of_five_test. Jis naudojamas visų naujų metodų patikrai. Kodui praėjus testavimą į konsolę yra išvedama eilutė "Visi rule of five ir operatoriu testai praejo.", jeigu programa yra terminuojama, reiškias program nepraėjo testavimų.

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Zmogus	15
Studentas< std::vector< int > >	9
Studentas< std::list< int > >	9
Studentas< std::deque< int > >	9
Studentas< GradeContainer >	9

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

[Studentas](#) < [GradeContainer](#) >

Studentą aprašanti šabloninė klasė 9

[Zmogus](#)

Abstrakti bazinė klasė žmogaus vardui ir pavardei saugoti 15

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

funkcijos.h	19
library.h	28
main.cpp	28
objektinis_patikrinimai.cpp	29
patikrinimai.h	30
rule_of_five_test.cpp	30

Chapter 5

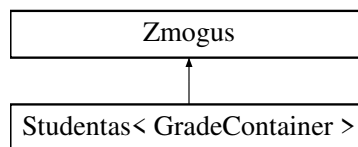
Class Documentation

5.1 Studentas< GradeContainer > Class Template Reference

Studentą aprašanti šabloninė klasė.

```
#include <funkcijos.h>
```

Inheritance diagram for Studentas< GradeContainer >:



Public Member Functions

- **Studentas** (const std::string &vardas, const std::string &pavarde, const GradeContainer &pazymiai, int egzaminas, double vidurkis, double mediana)
Sukuria studento objektą su visais pagrindiniais duomenimis.
- **Studentas** (const **Studentas** &other)
Kopijavimo konstruktorius.
- **Studentas** (**Studentas** &&other)
Perkėlimo konstruktorius.
- **Studentas** & **operator=** (const **Studentas** &other)
Kopijavimo priskyrimo operatorius.
- **Studentas** & **operator=** (**Studentas** &&other)
Perkėlimo priskyrimo operatorius.
- **~Studentas** ()
Sunaikina studento objektą ir išvalo jo laukus.
- GradeContainer & **pazymiai** ()
Grąžina modifikuojamą namų darbų pažymių konteinerį.
- const GradeContainer & **pazymiai** () const
Grąžina tik skaitymui skirtą namų darbų pažymių konteinerį.
- int **egzaminas** () const
Grąžina egzamino pažymį.
- void **setEgzaminas** (int egzaminas)
Nustato egzamino pažymį.
- double **vidurkis** () const
Grąžina galutinį įvertinimą pagal vidurkį.
- void **setVidurkis** (double vidurkis)
Nustato galutinį įvertinimą pagal vidurkį.

- double [mediana](#) () const
Grąžina galutinį įvertinimą pagal medianą.
- void [setMediana](#) (double mediana)
Nustato galutinį įvertinimą pagal medianą.
- std::string [tipas](#) () const override
Grąžina objekto tipą.

Public Member Functions inherited from [Zmogus](#)

- [Zmogus](#) (const std::string &vardas, const std::string &pavarde)
Sukuria žmogaus objektą su vardu ir pavarde.
- [Zmogus](#) (const [Zmogus](#) &other)
Kopijavimo konstruktorius.
- [Zmogus](#) ([Zmogus](#) &&other)
Perkėlimo konstruktorius.
- [Zmogus](#) & [operator=](#) (const [Zmogus](#) &other)
Kopijavimo priskyrimo operatorius.
- [Zmogus](#) & [operator=](#) ([Zmogus](#) &&other)
Perkėlimo priskyrimo operatorius.
- virtual [~Zmogus](#) ()
Virtualus destruktorius.
- const std::string & [vardas](#) () const
Grąžina žmogaus vardą.
- void [setVardas](#) (const std::string &vardas)
Nustato žmogaus vardą.
- const std::string & [pavarde](#) () const
Grąžina žmogaus pavardę.
- void [setPavarde](#) (const std::string &pavarde)
Nustato žmogaus pavardę.

Private Attributes

- GradeContainer [pazymiai](#)_
Namų darbų pažymiai.
- int [egzaminas](#)_
Egzamino pažymys.
- double [vidurkis](#)_
Galutinis įvertinimas pagal vidurkį.
- double [mediana](#)_
Galutinis įvertinimas pagal medianą.

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- std::string [vardas](#)_
Žmogaus vardas.
- std::string [pavarde](#)_
Žmogaus pavardė.

5.1.1 Detailed Description

```
template<typename GradeContainer>
class Studentas< GradeContainer >
```

Studentą aprašanti šabloninė klasė.

Template Parameters

<i>GradeContainer</i>	Konteinerio tipas namų darbų pažymiams saugoti.
-----------------------	---

Klasė saugo studento vardą, pavardę, namų darbų pažymius, egzamino rezultatą ir galutinius įvertinimus pagal vidurkį bei medianą.

Definition at line 122 of file [funkcijos.h](#).

5.1.2 Constructor & Destructor Documentation

5.1.2.1 Studentas() [1/3]

```
template<typename GradeContainer>
Studentas< GradeContainer >::Studentas (
    const std::string & vardas,
    const std::string & pavarde,
    const GradeContainer & pazymiai,
    int egzaminas,
    double vidurkis,
    double mediana) [inline]
```

Sukuria studento objektą su visais pagrindiniais duomenimis.

Parameters

<i>vardas</i>	Studento vardas.
<i>pavarde</i>	Studento pavardė.
<i>pazymiai</i>	Namų darbų pažymių konteineris.
<i>egzaminas</i>	Egzamino pažymys.
<i>vidurkis</i>	Galutinis įvertinimas pagal vidurkį.
<i>mediana</i>	Galutinis įvertinimas pagal medianą.

Definition at line 133 of file [funkcijos.h](#).

5.1.2.2 Studentas() [2/3]

```
template<typename GradeContainer>
Studentas< GradeContainer >::Studentas (
    const Studentas< GradeContainer > & other) [inline]
```

Kopijavimo konstruktorius.

Parameters

<i>other</i>	Kopijuojamas studentas.
--------------	-------------------------

Definition at line 145 of file [funkcijos.h](#).

5.1.2.3 Studentas() [3/3]

```
template<typename GradeContainer>
Studentas< GradeContainer >::Studentas (
    Studentas< GradeContainer > && other) [inline]
```

Perkėlimo konstruktorius.

Parameters

<i>other</i>	Studentas , iš kurio perkeliami duomenys.
--------------	---

Definition at line 156 of file [funkcijos.h](#).

5.1.2.4 ~Studentas()

```
template<typename GradeContainer>
Studentas< GradeContainer >::~~Studentas () [inline]
```

Sunaikina studento objektą ir išvalo jo laukus.
Definition at line 211 of file [funkcijos.h](#).

5.1.3 Member Function Documentation

5.1.3.1 egzaminas()

```
template<typename GradeContainer>
int Studentas< GradeContainer >::egzaminas () const [inline]
```

Grąžina egzamino pažymį.

Returns

Egzamino pažymys.

Definition at line 236 of file [funkcijos.h](#).

5.1.3.2 mediana()

```
template<typename GradeContainer>
double Studentas< GradeContainer >::mediana () const [inline]
```

Grąžina galutinį įvertinimą pagal medianą.

Returns

Galutinis įvertinimas pagal medianą.

Definition at line 260 of file [funkcijos.h](#).

5.1.3.3 operator=() [1/2]

```
template<typename GradeContainer>
Studentas & Studentas< GradeContainer >::operator= (
    const Studentas< GradeContainer > & other) [inline]
```

Kopijavimo priskyrimo operatorius.

Parameters

<i>other</i>	Kopijuojamas studentas.
--------------	-------------------------

Returns

Nuoroda į šį studentą.

Definition at line 175 of file [funkcijos.h](#).

5.1.3.4 operator=() [2/2]

```
template<typename GradeContainer>
Studentas & Studentas< GradeContainer >::operator= (
    Studentas< GradeContainer > && other) [inline]
```

Perkėlimo priskyrimo operatorius.

Parameters

<i>other</i>	Studentas , iš kurio perkeliami duomenys.
--------------	---

Returns

Nuoroda į šį studentą.

Definition at line 191 of file [funkcijos.h](#).

5.1.3.5 pazymiai() [1/2]

```
template<typename GradeContainer>
GradeContainer & Studentas< GradeContainer >::pazymiai () [inline]
```

Gražina modifikuojamą namų darbų pažymių konteinerį.

Returns

Nuoroda į pažymių konteinerį.

Definition at line 224 of file [funkcijos.h](#).

5.1.3.6 pazymiai() [2/2]

```
template<typename GradeContainer>
const GradeContainer & Studentas< GradeContainer >::pazymiai () const [inline]
```

Gražina tik skaitymui skirtą namų darbų pažymių konteinerį.

Returns

Konstantinė nuoroda į pažymių konteinerį.

Definition at line 230 of file [funkcijos.h](#).

5.1.3.7 setEgzaminas()

```
template<typename GradeContainer>
void Studentas< GradeContainer >::setEgzaminas (
    int egzaminas) [inline]
```

Nustato egzamino pažymį.

Parameters

<i>egzaminas</i>	Naujas egzamino pažymys.
------------------	--------------------------

Definition at line 242 of file [funkcijos.h](#).

5.1.3.8 setMediana()

```
template<typename GradeContainer>
void Studentas< GradeContainer >::setMediana (
    double mediana) [inline]
```

Nustato galutinį įvertinimą pagal medianą.

Parameters

<i>mediana</i>	Nauja galutinės medianos reikšmė.
----------------	-----------------------------------

Definition at line 266 of file [funkcijos.h](#).

5.1.3.9 setVidurkis()

```
template<typename GradeContainer>
void Studentas< GradeContainer >::setVidurkis (
    double vidurkis) [inline]
```

Nustato galutinį įvertinimą pagal vidurkį.

Parameters

<i>vidurkis</i>	Nauja galutinio vidurkio reikšmė.
-----------------	-----------------------------------

Definition at line 254 of file [funkcijos.h](#).

5.1.3.10 tipas()

```
template<typename GradeContainer>
std::string Studentas< GradeContainer >::tipas () const [inline], [override], [virtual]
```

Grąžina objekto tipą.

Returns

Tekstas "Studentas".

Implements [Zmogus](#).

Definition at line 272 of file [funkcijos.h](#).

5.1.3.11 vidurkis()

```
template<typename GradeContainer>
double Studentas< GradeContainer >::vidurkis () const [inline]
```

Grąžina galutinį įvertinimą pagal vidurkį.

Returns

Galutinis įvertinimas pagal vidurkį.

Definition at line 248 of file [funkcijos.h](#).

5.1.4 Member Data Documentation**5.1.4.1 egzaminas_**

```
template<typename GradeContainer>
int Studentas< GradeContainer >::egzaminas_ = 0 [private]
```

Egzamino pažymys.

Definition at line 278 of file [funkcijos.h](#).

5.1.4.2 mediana_

```
template<typename GradeContainer>
double Studentas< GradeContainer >::mediana_ = 0.0 [private]
```

Galutinis įvertinimas pagal medianą.

Definition at line 282 of file [funkcijos.h](#).

5.1.4.3 pazymiai_

```
template<typename GradeContainer>
GradeContainer Studentas< GradeContainer >::pazymiai_ [private]
Namų darbų pažymiai.
Definition at line 276 of file funkcijos.h.
```

5.1.4.4 vidurkis_

```
template<typename GradeContainer>
double Studentas< GradeContainer >::vidurkis_ = 0.0 [private]
Galutinis įvertinimas pagal vidurkį.
Definition at line 280 of file funkcijos.h.
```

The documentation for this class was generated from the following file:

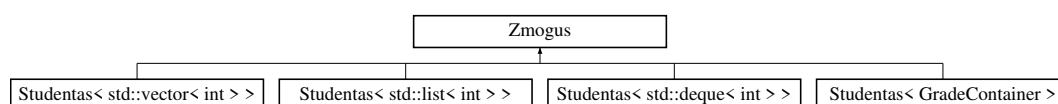
- funkcijos.h

5.2 Zmogus Class Reference

Abstrakti bazinė klasė žmogaus vardui ir pavardei saugoti.

```
#include <funkcijos.h>
```

Inheritance diagram for Zmogus:



Public Member Functions

- [Zmogus](#) (const std::string &vardas, const std::string &pavarde)
Sukuria žmogaus objektą su vardu ir pavarde.
- [Zmogus](#) (const [Zmogus](#) &other)
Kopijavimo konstruktorius.
- [Zmogus](#) ([Zmogus](#) &&other)
Perkėlimo konstruktorius.
- [Zmogus](#) & [operator=](#) (const [Zmogus](#) &other)
Kopijavimo priskyrimo operatorius.
- [Zmogus](#) & [operator=](#) ([Zmogus](#) &&other)
Perkėlimo priskyrimo operatorius.
- virtual [~Zmogus](#) ()
Virtualus destruktorius.
- const std::string & [vardas](#) () const
Grąžina žmogaus vardą.
- void [setVardas](#) (const std::string &vardas)
Nustato žmogaus vardą.
- const std::string & [pavarde](#) () const
Grąžina žmogaus pavardę.
- void [setPavarde](#) (const std::string &pavarde)
Nustato žmogaus pavardę.
- virtual std::string [tipas](#) () const =0
Grąžina konkretaus objekto tipą.

Protected Attributes

- `std::string vardas_`
Žmogaus vardas.
- `std::string pavarde_`
Žmogaus pavardė.

5.2.1 Detailed Description

Abstrakti bazinė klasė žmogaus vardui ir pavardei saugoti.

Klasė naudojama kaip bendras pagrindas konkretiems asmenų tipams. Ji turi virtualų metodą tipui nustatyti, todėl tiesiogiai objektų iš jos kurti negalima.

Definition at line 13 of file [funkcijos.h](#).

5.2.2 Constructor & Destructor Documentation

5.2.2.1 Zmogus() [1/3]

```
Zmogus::Zmogus (
    const std::string & vardas,
    const std::string & pavarde) [inline]
```

Sukuria žmogaus objektą su vardu ir pavarde.

Parameters

<i>vardas</i>	Žmogaus vardas.
<i>pavarde</i>	Žmogaus pavardė.

Definition at line 20 of file [funkcijos.h](#).

5.2.2.2 Zmogus() [2/3]

```
Zmogus::Zmogus (
    const Zmogus & other) [inline]
```

Kopijavimo konstruktorius.

Parameters

<i>other</i>	Kopijuojamas objektas.
--------------	------------------------

Definition at line 28 of file [funkcijos.h](#).

5.2.2.3 Zmogus() [3/3]

```
Zmogus::Zmogus (
    Zmogus && other) [inline]
```

Perkėlimo konstruktorius.

Parameters

<i>other</i>	Objektas, iš kurio perkeliama duomenys.
--------------	---

Definition at line 36 of file [funkcijos.h](#).

5.2.2.4 ~Zmogus()

```
virtual Zmogus::~Zmogus () [inline], [virtual]
```

Virtualus destruktorius.

Definition at line 72 of file [funkcijos.h](#).

5.2.3 Member Function Documentation

5.2.3.1 operator=() [1/2]

```
Zmogus & Zmogus::operator= (
    const Zmogus & other) [inline]
```

Kopijavimo priskyrimo operatorius.

Parameters

<i>other</i>	Kopijuojamas objektas.
--------------	------------------------

Returns

Nuoroda į šį objektą.

Definition at line 48 of file [funkcijos.h](#).

5.2.3.2 operator=() [2/2]

```
Zmogus & Zmogus::operator= (
    Zmogus && other) [inline]
```

Perkėlimo priskyrimo operatorius.

Parameters

<i>other</i>	Objektas, iš kurio perkeliama duomenys.
--------------	---

Returns

Nuoroda į šį objektą.

Definition at line 61 of file [funkcijos.h](#).

5.2.3.3 pavarde()

```
const std::string & Zmogus::pavarde () const [inline]
```

Grąžina žmogaus pavardę.

Returns

Konstantinė nuoroda į pavardę.

Definition at line 93 of file [funkcijos.h](#).

5.2.3.4 setPavarde()

```
void Zmogus::setPavarde (
    const std::string & pavarde) [inline]
```

Nustato žmogaus pavardę.

Parameters

<i>pavarde</i>	Nauja pavardė.
----------------	----------------

Definition at line 99 of file [funkcijos.h](#).

5.2.3.5 setVardas()

```
void Zmogus::setVardas (
    const std::string & vardas) [inline]
```

Nustato žmogaus vardą.

Parameters

<i>vardas</i>	Naujas vardas.
---------------	----------------

Definition at line 87 of file [funkcijos.h](#).

5.2.3.6 tipas()

```
virtual std::string Zmogus::tipas () const [pure virtual]
```

Grąžina konkretaus objekto tipą.

Returns

Tipo pavadinimas tekstu.

Implemented in [Studentas< GradeContainer >](#), [Studentas< std::deque< int > >](#), [Studentas< std::list< int > >](#), and [Studentas< std::vector< int > >](#).

5.2.3.7 vardas()

```
const std::string & Zmogus::vardas () const [inline]
```

Grąžina žmogaus vardą.

Returns

Konstantinė nuoroda į vardą.

Definition at line 81 of file [funkcijos.h](#).

5.2.4 Member Data Documentation

5.2.4.1 pavarde_

```
std::string Zmogus::pavarde_ [protected]
```

Žmogaus pavardė.

Definition at line 111 of file [funkcijos.h](#).

5.2.4.2 vardas_

```
std::string Zmogus::vardas_ [protected]
```

Žmogaus vardas.

Definition at line 109 of file [funkcijos.h](#).

The documentation for this class was generated from the following file:

- [funkcijos.h](#)

Chapter 6

File Documentation

6.1 funkcijos.h

```
00001 #ifndef STUDENTAS_H
00002 #define STUDENTAS_H
00003
00004 #include "library.h"
00005 #include "patikrinimai.h"
00006
00013 class Zmogus {
00014 public:
00020     Zmogus(const std::string& vardas, const std::string& pavarde)
00021         : vardas_(vardas),
00022           pavarde_(pavarde) {}
00023
00028     Zmogus(const Zmogus& other)
00029         : vardas_(other.vardas_),
00030           pavarde_(other.pavarde_) {}
00031
00036     Zmogus(Zmogus&& other)
00037         : vardas_(std::move(other.vardas_)),
00038           pavarde_(std::move(other.pavarde_)) {
00039         other.vardas_.clear();
00040         other.pavarde_.clear();
00041     }
00042
00048     Zmogus& operator=(const Zmogus& other) {
00049         if (this != &other) {
00050             vardas_ = other.vardas_;
00051             pavarde_ = other.pavarde_;
00052         }
00053         return *this;
00054     }
00055
00061     Zmogus& operator=(Zmogus&& other) {
00062         if (this != &other) {
00063             vardas_ = std::move(other.vardas_);
00064             pavarde_ = std::move(other.pavarde_);
00065         }
00066         return *this;
00067     }
00068
00072     virtual ~Zmogus() {
00073         vardas_.clear();
00074         pavarde_.clear();
00075     }
00076
00081     const std::string& vardas() const { return vardas_; }
00082
00087     void setVardas(const std::string& vardas) { vardas_ = vardas; }
00088
00093     const std::string& pavarde() const { return pavarde_; }
00094
00099     void setPavarde(const std::string& pavarde) { pavarde_ = pavarde; }
00100
00105     virtual std::string tipas() const = 0;
00106
00107 protected:
00109     std::string vardas_;
00111     std::string pavarde_;
00112 };
00113
00121 template <typename GradeContainer>
00122 class Studentas : public Zmogus {
```

```

00123 public:
00133     Studentas(const std::string& vardas, const std::string& pavarde, const GradeContainer& pazymiai,
00134               int egzaminas, double vidurkis, double mediana)
00135     : Zmogus(vardas, pavarde),
00136       pazymiai_(pazymiai),
00137       egzaminas_(egzaminas),
00138       vidurkis_(vidurkis),
00139       mediana_(mediana) {}
00140
00145     Studentas(const Studentas& other)
00146     : Zmogus(other),
00147       pazymiai_(other.pazymiai_),
00148       egzaminas_(other.egzaminas_),
00149       vidurkis_(other.vidurkis_),
00150       mediana_(other.mediana_) {}
00151
00156     Studentas(Studentas&& other)
00157     : Zmogus(std::move(other)),
00158       pazymiai_(std::move(other.pazymiai_)),
00159       egzaminas_(other.egzaminas_),
00160       vidurkis_(other.vidurkis_),
00161       mediana_(other.mediana_) {
00162         other.vardas_.clear();
00163         other.pavarde_.clear();
00164         other.pazymiai_.clear();
00165         other.egzaminas_ = 0;
00166         other.vidurkis_ = 0.0;
00167         other.mediana_ = 0.0;
00168     }
00169
00175     Studentas& operator=(const Studentas& other) {
00176         if (this != &other) {
00177             Zmogus::operator=(other);
00178             pazymiai_ = other.pazymiai_;
00179             egzaminas_ = other.egzaminas_;
00180             vidurkis_ = other.vidurkis_;
00181             mediana_ = other.mediana_;
00182         }
00183         return *this;
00184     }
00185
00191     Studentas& operator=(Studentas&& other) {
00192         if (this != &other) {
00193             Zmogus::operator=(std::move(other));
00194             pazymiai_ = std::move(other.pazymiai_);
00195             egzaminas_ = other.egzaminas_;
00196             vidurkis_ = other.vidurkis_;
00197             mediana_ = other.mediana_;
00198             other.vardas_.clear();
00199             other.pavarde_.clear();
00200             other.pazymiai_.clear();
00201             other.egzaminas_ = 0;
00202             other.vidurkis_ = 0.0;
00203             other.mediana_ = 0.0;
00204         }
00205         return *this;
00206     }
00207
00211     ~Studentas() {
00212         vardas_.clear();
00213         pavarde_.clear();
00214         pazymiai_.clear();
00215         egzaminas_ = 0;
00216         vidurkis_ = 0.0;
00217         mediana_ = 0.0;
00218     }
00219
00224     GradeContainer& pazymiai() { return pazymiai_; }
00225
00230     const GradeContainer& pazymiai() const { return pazymiai_; }
00231
00236     int egzaminas() const { return egzaminas_; }
00237
00242     void setEgzaminas(int egzaminas) { egzaminas_ = egzaminas; }
00243
00248     double vidurkis() const { return vidurkis_; }
00249
00254     void setVidurkis(double vidurkis) { vidurkis_ = vidurkis; }
00255
00260     double mediana() const { return mediana_; }
00261
00266     void setMediana(double mediana) { mediana_ = mediana; }
00267
00272     std::string tipas() const override { return "Studentas"; }
00273
00274 private:
00276     GradeContainer pazymiai_;

```

```

00278     int egzaminas_ = 0;
00280     double vidurkis_ = 0.0;
00282     double mediana_ = 0.0;
00283 };
00284
00286 using VectorStudent = Studentas<std::vector<int>>;
00288 using ListStudent = Studentas<std::list<int>>;
00290 using DequeStudent = Studentas<std::deque<int>>;
00291
00293 using VectorContainer = std::vector<VectorStudent>;
00295 using ListContainer = std::list<ListStudent>;
00297 using DequeContainer = std::deque<DequeStudent>;
00298
00306 template <typename GradeContainer>
00307 std::ostream& operator<>(std::ostream& out, const Studentas<GradeContainer>& studentas) {
00308     out << studentas.vardas() << " "
00309         << studentas.pavarde() << " "
00310         << studentas.egzaminas() << " "
00311         << studentas.pazymiai().size();
00312
00313     for (int pazymis : studentas.pazymiai()) {
00314         out << " " << pazymis;
00315     }
00316
00317     return out;
00318 }
00319
00327 template <typename GradeContainer>
00328 std::istream& operator<>(std::istream& in, Studentas<GradeContainer>& studentas) {
00329     std::string vardas;
00330     std::string pavarde;
00331     int egzaminas = 0;
00332     int pazymiu_kiekis = 0;
00333
00334     if (!(in >> vardas >> pavarde >> egzaminas >> pazymiu_kiekis)) {
00335         return in;
00336     }
00337
00338     studentas.setVardas(vardas);
00339     studentas.setPavarde(pavarde);
00340     studentas.setEgzaminas(egzaminas);
00341     studentas.pazymiai().clear();
00342
00343     for (int i = 0; i < pazymiu_kiekis; ++i) {
00344         int pazymis = 0;
00345         if (!(in >> pazymis)) {
00346             return in;
00347         }
00348         studentas.pazymiai().push_back(pazymis);
00349     }
00350
00351     return in;
00352 }
00353
00354
00363 template <typename T, typename Allocator, typename Compare>
00364 void sort_container(std::list<T, Allocator>& container, Compare comp) {
00365     container.sort(comp);
00366 }
00367
00375 template <typename Container, typename Compare>
00376 void sort_container(Container& container, Compare comp) {
00377     std::sort(container.begin(), container.end(), comp);
00378 }
00379
00386 template <typename GradeContainer>
00387 double skaiciuoti_mediana(GradeContainer& paz) {
00388     if (paz.empty()) {
00389         return 0.0;
00390     }
00391
00392     if constexpr (std::is_same_v<GradeContainer, std::list<int>>) {
00393         paz.sort(std::less<int>{});
00394     } else {
00395         std::sort(paz.begin(), paz.end(), std::less<int>{});
00396     }
00397
00398     const std::size_t dydis = paz.size();
00399     auto mid = paz.begin();
00400     std::advance(mid, static_cast<long>(dydis / 2));
00401     if (dydis % 2 == 0) {
00402         auto left = mid;
00403         --left;
00404         return (*left + *mid) / 2.0;
00405     }
00406     return static_cast<double>(*mid);
00407 }

```

```

00408
00415 template <typename Student>
00416 void paz_sk(Student& temp, int& m) {
00417     std::string input;
00418     while (true) {
00419         std::cout << "Kiek pazymiu turi " << temp.vardas() << " " << temp.pavarde() << "? ";
00420         if (!read_input(input)) {
00421             return;
00422         }
00423         m = validation(input);
00424         if (m == 0) {
00425             std::cout << "Ne skaicius!" << std::endl;
00426         } else if (m < 1) {
00427             std::cout << "Iveskite tinkama sk." << std::endl;
00428         } else {
00429             break;
00430         }
00431     }
00432 }
00433
00440 template <typename Student>
00441 void paz_ivestis_ranka(Student& temp, int m) {
00442     std::string input;
00443     for (int j = 0; j < m; ++j) {
00444         std::cout << "Iveskite " << j + 1 << " pazymi: ";
00445         if (!read_input(input)) {
00446             return;
00447         }
00448         const int pazymis = validation(input);
00449         if (pazymis > 10 || pazymis < 1) {
00450             std::cout << "Netinkamas sk. Bandykite dar karta" << std::endl;
00451             --j;
00452             continue;
00453         }
00454         temp.pazymiai().push_back(pazymis);
00455     }
00456 }
00457
00463 template <typename Student>
00464 void egz_ivestis_ranka(Student& temp) {
00465     std::string input;
00466     while (true) {
00467         std::cout << "Koks yra " << temp.vardas() << " " << temp.pavarde() << " egzamino rezultatas? ";
00468         if (!read_input(input)) {
00469             return;
00470         }
00471         const int egz = validation(input);
00472         if (egz > 10 || egz < 1) {
00473             std::cout << "Netinkamas sk. Bandykite dar karta" << std::endl;
00474             continue;
00475         }
00476         temp.setEgzaminas(egz);
00477         break;
00478     }
00479 }
00480
00487 template <typename Student>
00488 void vardu_ivedimas_ranka(Student& temp, std::size_t current_count) {
00489     while (true) {
00490         std::string vardas;
00491         std::string pavarde;
00492         std::cout << "Koks yra " << current_count + 1 << " studento vardas ir pavarde? ";
00493         if (!(std::cin >> vardas >> pavarde)) {
00494             if (std::cin.eof()) {
00495                 std::cout << "Ivestis nutraukta." << std::endl;
00496                 return;
00497             }
00498             std::cin.clear();
00499             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00500             std::cout << "Netinkama ivestis. Bandykite dar karta." << std::endl;
00501             continue;
00502         }
00503         if (valid_name(vardas) && valid_name(pavarde)) {
00504             temp.setVardas(vardas);
00505             temp.setPavarde(pavarde);
00506             break;
00507         }
00508         std::cout << "Iveskite tinkama varda ir pavarde" << std::endl;
00509     }
00510 }
00511
00518 template <typename Student>
00519 void paz_ivestis_random(Student& temp, int m) {
00520     std::random_device rd;
00521     std::mt19937 gen(rd());
00522     std::uniform_int_distribution<> dist(1, 10);
00523     for (int i = 0; i < m; ++i) {

```

```

00524         temp.pazymiai().push_back(dist(gen));
00525     }
00526 }
00527
00533 template <typename Student>
00534 void egz_ivestis_random(Student& temp) {
00535     std::random_device rd;
00536     std::mt19937 gen(rd());
00537     std::uniform_int_distribution<> dist(1, 10);
00538     temp.setEgzaminas(dist(gen));
00539 }
00540
00546 template <typename Student>
00547 void vardu_ivedimas_random(Student& temp) {
00548     static const std::vector<std::string> vard = {
00549         "Artur", "Simas", "Romas", "Patrikas", "Rokas",
00550         "Ignas", "Tomas", "Rugile", "Aiste", "Martynas"
00551     };
00552     static const std::vector<std::string> pav = {
00553         "Pavardenis1", "Pavardenis2", "Pavardenis3", "Pavardenis4", "Pavardenis5",
00554         "Pavardenis6", "Pavardenis7", "Pavardenis8", "Pavardenis9", "Pavardenis10"
00555     };
00556     std::random_device rd;
00557     std::mt19937 gen(rd());
00558     std::uniform_int_distribution<> dist(0, 9);
00559     const int r = dist(gen);
00560     temp.setVardas(vard[r]);
00561     temp.setPavarde(pav[r]);
00562 }
00563 }
00564
00570 template <typename StudentContainer>
00571 void vidurkis(StudentContainer& stud) {
00572     for (auto& studentas : stud) {
00573         double sum = 0.0;
00574         for (int pazymis : studentas.pazymiai()) {
00575             sum += pazymis;
00576         }
00577         studentas.setVidurkis((sum / static_cast<double>(studentas.pazymiai().size())) * 0.4
00578             + studentas.egzaminas() * 0.6);
00579     }
00580 }
00581
00587 template <typename StudentContainer>
00588 void mediana(StudentContainer& stud) {
00589     for (auto& studentas : stud) {
00590         studentas.setMediana(skaiciuoti_mediana(studentas.pazymiai()));
00591     }
00592 }
00593
00599 template <typename StudentContainer>
00600 void isvestis(StudentContainer& stud) {
00601     std::string input;
00602     while (true) {
00603         std::cout << "Ka noretumet pamatyti? Mediana - 1, arba Vidurki - 2 ";
00604         if (!read_input(input)) {
00605             return;
00606         }
00607         const int choice = validation(input);
00608         if (choice != 1 && choice != 2) {
00609             std::cout << "Iveskite tinkama sk!" << std::endl;
00610             continue;
00611         }
00612         if (choice == 1) {
00613             std::cout << std::left << std::setw(15) << "Vardas" << std::setw(15) << "Pavarde"
00614                 << std::setw(20) << "Galutinis (Med.)" << std::endl;
00615         } else {
00616             std::cout << std::left << std::setw(15) << "Vardas" << std::setw(15) << "Pavarde"
00617                 << std::setw(20) << "Galutinis (Vid.)" << std::endl;
00618         }
00619         std::cout <<
00620         "-----" << std::endl;
00621         for (const auto& studentas : stud) {
00622             std::cout << std::fixed << std::left << std::setw(15) << studentas.vardas()
00623                 << std::setw(15) << studentas.pavarde() << std::setw(9) << std::setprecision(2)
00624                 << (choice == 1 ? studentas.mediana() : studentas.vidurkis()) << std::endl;
00625         }
00626         return;
00627     }
00628 }
00629 }
00630
00637 template <typename StudentContainer>
00638 void rusiavimas(StudentContainer& stud, double& laikas) {
00639     std::string input;
00640     while (true) {

```

```

00641         std::cout << "Rusiukite studentus pagal: 1 - vardas, 2 - pavarde, 3 - vidurki, 4 - mediana ";
00642         if (!read_input(input)) {
00643             return;
00644         }
00645
00646         const auto pradzia = std::chrono::high_resolution_clock::now();
00647         const int choice = validation(input);
00648         if (choice == 1) {
00649             sort_container(stud, [](const auto& a, const auto& b) { return a.vardas() < b.vardas();
00650         });
00651         } else if (choice == 2) {
00652             sort_container(stud, [](const auto& a, const auto& b) { return a.pavarde() < b.pavarde();
00653         });
00654         } else if (choice == 3) {
00655             sort_container(stud, [](const auto& a, const auto& b) { return a.vidurkis() <
00656         b.vidurkis(); });
00657         } else if (choice == 4) {
00658             sort_container(stud, [](const auto& a, const auto& b) { return a.mediana() < b.mediana();
00659         });
00660         } else {
00661             std::cout << "Iveskite tinkama sk!" << std::endl;
00662             continue;
00663         }
00664
00665         const auto pabaiga = std::chrono::high_resolution_clock::now();
00666         const std::chrono::duration<double, std::milli> trukme = pabaiga - pradzia;
00667         laikas += trukme.count();
00668         return;
00669     }
00670 }
00671
00672 inline void failu_kurimas(const std::string& name, int zmones, int m, double& laikas) {
00673     std::random_device rd;
00674     std::mt19937 gen(rd());
00675     std::uniform_int_distribution<> dist(1, 10);
00676
00677     const auto pradzia = std::chrono::high_resolution_clock::now();
00678
00679     std::ofstream failas(name);
00680     failas << "vardas pavarde ";
00681     for (int i = 0; i < m; ++i) {
00682         failas << " ND" << i + 1;
00683     }
00684     failas << " egz" << std::endl;
00685     for (int i = 0; i < zmones; ++i) {
00686         failas << "vardas" << i + 1 << " pavarde" << i + 1;
00687         for (int j = 0; j < m; ++j) {
00688             failas << " " << dist(gen);
00689         }
00690         failas << " " << dist(gen) << std::endl;
00691     }
00692
00693     const auto pabaiga = std::chrono::high_resolution_clock::now();
00694     const std::chrono::duration<double, std::milli> trukme = pabaiga - pradzia;
00695     std::cout << zmones << " Rasymas uztruko " << trukme.count() << " ms" << std::endl;
00696     laikas += trukme.count();
00697 }
00698
00699 template <typename StudentContainer>
00700 void isvestis_failas(StudentContainer& stud) {
00701     std::string input;
00702     while (true) {
00703         std::cout << "Duomenis rasyti: 1 - i konsole, 2 - i atskira faila: ";
00704         if (!read_input(input)) {
00705             return;
00706         }
00707
00708         const int choice = validation(input);
00709         if (choice != 1 && choice != 2) {
00710             std::cout << "Iveskite tinkama sk!" << std::endl;
00711             continue;
00712         }
00713
00714         std::ostream* out = &std::cout;
00715         std::ofstream rezfailas;
00716         if (choice == 2) {
00717             rezfailas.open("rezultatai.txt");
00718             out = &rezfailas;
00719         }
00720
00721         *out << std::left << std::setw(15) << "Vardas" << std::setw(15) << "Pavarde"
00722         << std::setw(20) << "Galutinis (Vid.)" << std::setw(10) << "Galutinis (Med.)" << std::endl;
00723         *out << "-----" << std::endl;
00724
00725         for (const auto& studentas : stud) {
00726             *out << std::fixed << std::left << std::setw(15) << studentas.vardas()
00727             << std::setw(15) << studentas.pavarde() << std::setw(9) << std::setprecision(2)

```

```

00735         « studentas.vidurkis() « std::setw(9) « studentas.mediana() « std::endl;
00736     }
00737     return;
00738 }
00739 }
00740
00741 template <typename StudentContainer>
00742 void skaitymas(StudentContainer& stud, const std::string& input, double& laikas) {
00743     using Student = typename StudentContainer::value_type;
00744
00745     std::string line;
00746     int pazymis = 0;
00747
00748     try {
00749         const std::filesystem::path failo_kelias = input;
00750         if (!std::filesystem::exists(failo_kelias)) {
00751             throw std::runtime_error("Toks failas nurodytame aplanke nerastas!");
00752         }
00753         if (!std::filesystem::is_regular_file(failo_kelias)) {
00754             throw std::runtime_error("Nurodytas kelias nera failas!");
00755         }
00756
00757         const auto pradzia = std::chrono::high_resolution_clock::now();
00758         std::ifstream duomfailas(input);
00759         if (!duomfailas.is_open()) {
00760             throw std::runtime_error("Nepavyko atidaryti failo!");
00761         }
00762         if (duomfailas.peek() == std::ifstream::traits_type::eof()) {
00763             throw std::runtime_error("Failas tuscias!");
00764         }
00765
00766         std::stringstream buffer;
00767         buffer < duomfailas.rdbuf();
00768
00769         std::getline(buffer, line);
00770         while (std::getline(buffer, line)) {
00771             if (line.empty()) {
00772                 continue;
00773             }
00774
00775             Student temp("", "", {}, 0, 0.0, 0.0);
00776             std::istringstream laik(line);
00777             std::string vardas;
00778             std::string pavarde;
00779             if (!(laik » vardas » pavarde)) {
00780                 throw std::runtime_error("Netinkamas failo formatas.");
00781             }
00782             temp.setVardas(vardas);
00783             temp.setPavarde(pavarde);
00784
00785             while (laik » pazymis) {
00786                 if (pazymis < 1 || pazymis > 10) {
00787                     throw std::runtime_error("Pazymiai faile turi buti nuo 1 iki 10.");
00788                 }
00789                 temp.pazymiai().push_back(pazymis);
00790             }
00791
00792             if (laik.fail() && !laik.eof()) {
00793                 throw std::runtime_error("Netinkamas pazymio formatas faile.");
00794             }
00795             if (temp.pazymiai().empty()) {
00796                 throw std::runtime_error("Studentas neturi nei vieno pazymio.");
00797             }
00798
00799             temp.setEgzaminas(temp.pazymiai().back());
00800             temp.pazymiai().pop_back();
00801             if (temp.pazymiai().empty()) {
00802                 throw std::runtime_error("Truksta namu darbu pazymiu.");
00803             }
00804
00805             stud.push_back(temp);
00806         }
00807
00808         const auto pabaiga = std::chrono::high_resolution_clock::now();
00809         const std::chrono::duration<double, std::milli> trukme = pabaiga - pradzia;
00810         std::cout < "Skaitymas uztruko " < trukme.count() < " ms" < std::endl;
00811         laikas += trukme.count();
00812     } catch (const std::filesystem::filesystem_error& e) {
00813         std::cout < "Failu sistemos klaida: " < e.what() < std::endl;
00814     } catch (const std::exception& e) {
00815         std::cout < e.what() < std::endl;
00816     }
00817 }
00818
00819 template <typename StudentContainer>
00820 void skirstymas(StudentContainer& stud, StudentContainer& maladiiec, StudentContainer& lopai, double& laikas) {

```

```

00836     const auto pradzia = std::chrono::high_resolution_clock::now();
00837     (void)stud;
00838
00839     auto border = std::partition(maladiec.begin(), maladiec.end(), [](const auto& studentas) {
00840         return studentas.vidurkis() >= 5;
00841     });
00842
00843     for (auto it = border; it != maladiec.end(); ++it) {
00844         lopai.push_back(*it);
00845     }
00846     maladiec.erase(border, maladiec.end());
00847
00848     const auto pabaiga = std::chrono::high_resolution_clock::now();
00849     const std::chrono::duration<double, std::milli> trukme = pabaiga - pradzia;
00850     std::cout << "Skirstymas i 2 konteinerius truko " << trukme.count() << " ms" << std::endl;
00851     laikas += trukme.count();
00852 }
00853
00861 template <typename StudentContainer>
00862 void rasymas(const StudentContainer& a, const std::string& name, double& laikas) {
00863     const auto pradzia = std::chrono::high_resolution_clock::now();
00864
00865     std::ofstream rezfailas(name);
00866     rezfailas << std::left << std::setw(15) << "Vardas" << std::setw(15) << "Pavarde"
00867         << std::setw(20) << "Galutinis (Vid.)" << std::setw(10) << "Galutinis (Med.)" << std::endl;
00868     rezfailas << "-----"
00869     << std::endl;
00870     for (const auto& studentas : a) {
00871         rezfailas << std::fixed << std::left << std::setw(15) << studentas.vardas()
00872             << std::setw(15) << studentas.pavarde() << std::setw(9) << std::setprecision(2)
00873             << studentas.vidurkis() << std::setw(9) << studentas.mediana() << std::endl;
00874     }
00875
00876     const auto pabaiga = std::chrono::high_resolution_clock::now();
00877     const std::chrono::duration<double, std::milli> trukme = pabaiga - pradzia;
00878     std::cout << name << " Rasymas truko " << trukme.count() << " ms" << std::endl;
00879     laikas += trukme.count();
00880 }
00881
00889 template <typename StudentContainer>
00890 void testavimas(StudentContainer& stud, StudentContainer& maladiec, StudentContainer& lopai, double&
    laikas) {
00891     vidurkis(maladiec);
00892     mediana(maladiec);
00893     sort_ascending(maladiec, laikas);
00894     skirstymas(stud, maladiec, lopai, laikas);
00895     rasymas(maladiec, "maladiec.txt", laikas);
00896     rasymas(lopai, "lopai.txt", laikas);
00897     std::cout << "Darbas su failu uztruko " << laikas << " ms" << std::endl;
00898     std::cout << std::endl;
00899 }
00900
00909 template <typename StudentContainer>
00910 void tyrimai_5(StudentContainer& stud, StudentContainer& maladiec, StudentContainer& lopai, double&
    laikas) {
00911     while (true) {
00912         std::string ivestis;
00913         std::cout << "Kuri faila norite skaityti? 1 - studentai1000.txt, 2 - studentai10000.txt, 3 -
00914 studentai100000.txt, 4 - studentai1000000.txt, 5 - studentai10000000.txt, 6 - baigti ";
00915         if (!read_input(ivistis)) {
00916             return;
00917         }
00918         const int p = validation(ivistis);
00919         if (p == 6) {
00920             break;
00921         }
00922
00923         std::string failo_vardas;
00924         if (p == 1) {
00925             failo_vardas = "studentai1000.txt";
00926         } else if (p == 2) {
00927             failo_vardas = "studentai10000.txt";
00928         } else if (p == 3) {
00929             failo_vardas = "studentai100000.txt";
00930         } else if (p == 4) {
00931             failo_vardas = "studentai1000000.txt";
00932         } else if (p == 5) {
00933             failo_vardas = "studentai10000000.txt";
00934         } else {
00935             std::cout << "Iveskite tinkama sk. " << std::endl;
00936             continue;
00937         }
00938
00939         stud.clear();
00940         maladiec.clear();
00941         lopai.clear();

```

```

00942         laikas = 0;
00943
00944         skaitymas(maladiec, failo_vardas, laikas);
00945         testavimas(stud, maladiec, lopai, laikas);
00946     }
00947 }
00948
00949 template <typename StudentContainer>
00950 int run_program(const std::string& konteinerio_pavadinimas) {
00951     using Student = typename StudentContainer::value_type;
00952
00953     StudentContainer stud;
00954     StudentContainer maladiec;
00955     StudentContainer lopai;
00956     std::string input;
00957     double b = 0.0;
00958     double laikas = 0.0;
00959
00960     std::cout << "Naudojamas konteineris: " << konteinerio_pavadinimas << std::endl;
00961     std::cout << "Studentu Vardu ir pazymiu ivedimu sistema, skirta medianos bei vidurkio
00962     apskaiciavimui" << std::endl;
00963     while (true) {
00964         Student temp("", "", {}, 0, 0.0, 0.0);
00965         int m = 0;
00966         while (true) {
00967             std::cout << "1 - ranka, 2 - generuoti tik pazymius, 3 - generuoti studentu vardus,
00968             pavardes ir pazymius, 4 - skaityti duomenis is failo, 5 - generuoti failus, 6 - baigti darba: ";
00969             if (!read_input(input)) {
00970                 return 0;
00971             }
00972
00973             const int choice = validation(input);
00974             if (choice == 1) {
00975                 vardu_ivedimas_ranka(temp, stud.size());
00976                 if (temp.vardas().empty() || temp.pavarde().empty()) {
00977                     break;
00978                 }
00979                 paz_sk(temp, m);
00980                 if (m < 1) {
00981                     break;
00982                 }
00983                 paz_ivestis_ranka(temp, m);
00984                 if (temp.pazymiai().size() != static_cast<std::size_t>(m)) {
00985                     break;
00986                 }
00987                 egz_ivestis_ranka(temp);
00988                 if (temp.egzaminas() == 0) {
00989                     break;
00990                 }
00991                 stud.push_back(temp);
00992             } else if (choice == 2) {
00993                 vardu_ivedimas_ranka(temp, stud.size());
00994                 if (temp.vardas().empty() || temp.pavarde().empty()) {
00995                     break;
00996                 }
00997                 paz_sk(temp, m);
00998                 if (m < 1) {
00999                     break;
01000                 }
01001                 paz_ivestis_random(temp, m);
01002                 egz_ivestis_random(temp);
01003                 stud.push_back(temp);
01004             } else if (choice == 3) {
01005                 vardu_ivedimas_random(temp);
01006                 paz_sk(temp, m);
01007                 if (m < 1) {
01008                     break;
01009                 }
01010                 paz_ivestis_random(temp, m);
01011                 egz_ivestis_random(temp);
01012                 stud.push_back(temp);
01013             } else if (choice == 4) {
01014                 tyrimai_5(stud, maladiec, lopai, laikas);
01015                 break;
01016             } else if (choice == 5) {
01017                 m = paz_sk();
01018                 failu_kurimas("studentail000.txt", 1000, m, b);
01019                 failu_kurimas("studentail0000.txt", 10000, m, b);
01020                 failu_kurimas("studentail00000.txt", 100000, m, b);
01021                 failu_kurimas("studentail000000.txt", 1000000, m, b);
01022                 failu_kurimas("studentail0000000.txt", 10000000, m, b);
01023                 std::cout << std::endl;
01024                 tyrimai_5(stud, maladiec, lopai, laikas);
01025                 break;
01026             } else if (choice == 6) {
01027                 std::cout << "Sekmingai baigete studentu duomenu ivedima!" << std::endl;
01028                 break;
01029             }
01030         }
01031     }
01032 }

```

```

01033         } else {
01034             std::cout << "Iveskite tinkama sk!" << std::endl;
01035         }
01036     }
01037     break;
01038 }
01039
01040 if (stud.empty()) {
01041     return 0;
01042 }
01043
01044 vidurkis(stud);
01045 mediana(stud);
01046 if (stud.size() > 1) {
01047     rusiavimas(stud, b);
01048 }
01049 isvestis_failas(stud);
01050 return 0;
01051 }
01052
01059 template <typename Container>
01060 void sort_ascending(Container& c, double& laikas) {
01061     const auto pradzia = std::chrono::high_resolution_clock::now();
01062     if constexpr (std::is_same_v<Container, std::list<typename Container::value_type> > {
01063         c.sort([](const auto& a, const auto& b) {
01064             return a.vidurkis() < b.vidurkis();
01065         });
01066     } else {
01067         std::sort(c.begin(), c.end(), [](const auto& a, const auto& b) {
01068             return a.vidurkis() < b.vidurkis();
01069         });
01070     }
01071     const auto pabaiga = std::chrono::high_resolution_clock::now();
01072     const std::chrono::duration<double, std::milli> trukme = pabaiga - pradzia;
01073     std::cout << "Sort truko " << trukme.count() << std::endl;
01074     laikas += trukme.count();
01075 }
01076
01077 #endif

```

6.2 library.h

```

00001 #ifndef LIBRARY_H
00002 #define LIBRARY_H
00003
00004 #include <algorithm>
00005 #include <chrono>
00006 #include <cctype>
00007 #include <cstdint>
00008 #include <deque>
00009 #include <filesystem>
00010 #include <fstream>
00011 #include <iomanip>
00012 #include <iostream>
00013 #include <iterator>
00014 #include <limits>
00015 #include <list>
00016 #include <random>
00017 #include <sstream>
00018 #include <string>
00019 #include <type_traits>
00020 #include <utility>
00021 #include <vector>
00022
00023 #endif

```

6.3 main.cpp

```

00001 #include "funkcijos.h"
00002 #include "library.h"
00003 #include "patikrinimai.h"
00004
00013 int main() {
00014     std::string input;
00015
00016     while (true) {
00017         std::cout << "Pasirinkite kontaineri: 1 - vector, 2 - list, 3 - deque: ";
00018         if (!read_input(input)) {
00019             return 0;
00020         }
00021     }

```

```

00022         const int choice = validation(input);
00023         if (choice == 1) {
00024             return run_program<VectorContainer>("vector");
00025         }
00026         if (choice == 2) {
00027             return run_program<ListContainer>("list");
00028         }
00029         if (choice == 3) {
00030             return run_program<DequeContainer>("deque");
00031         }
00032
00033         std::cout << "Iveskite tinkama sk!" << std::endl;
00034     }
00035 }

```

6.4 objektinis_patikrinimai.cpp

```

00001
00002 #include "library.h"
00003 #include "patikrinimai.h"
00004
00005 bool valid_name(const std::string& s) {
00006     if (s.empty()) {
00007         return false;
00008     }
00009
00010     for (unsigned char c : s) {
00011         if (!std::isalpha(c) && c != '-') {
00012             return false;
00013         }
00014     }
00015     return true;
00016 }
00017
00018 int validation(const std::string& a) {
00019     try {
00020         return std::stoi(a);
00021     } catch (const std::exception&) {
00022         return 0;
00023     }
00024 }
00025
00026 bool read_input(std::string& input) {
00027     try {
00028         if (std::cin >> input) {
00029             return true;
00030         }
00031
00032         if (std::cin.eof()) {
00033             std::cout << "Ivestis nutraukta." << std::endl;
00034             return false;
00035         }
00036
00037         std::cin.clear();
00038         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00039         std::cout << "Netinkama ivestis. Bandykite dar karta." << std::endl;
00040     }
00041     catch (const std::exception&) {
00042         std::cin.clear();
00043         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00044         std::cout << "Klaida skaitant ivesti. Bandykite dar karta." << std::endl;
00045     }
00046
00047     return false;
00048 }
00049
00050 int paz_sk() {
00051     std::string input;
00052     int pazsk = 0;
00053     while (true) {
00054         std::cout << "Kiek pazymiu tures studentai sarase? ";
00055         std::cin >> input;
00056         pazsk = validation(input);
00057         if (pazsk > 0) {
00058             return pazsk;
00059         }
00060         std::cout << "iveskite tinkama sk! " << std::endl;
00061     }
00062 }

```

6.5 patikrinimai.h

```

00001 #ifndef PATIKRINIMAI_H
00002 #define PATIKRINIMAI_H
00003
00004 #include "library.h"
00005
00011 bool valid_name(const std::string& s);
00012
00018 int validation(const std::string& a);
00019
00025 bool read_input(std::string& input);
00026
00031 int paz_sk();
00032
00033 #endif

```

6.6 rule_of_five_test.cpp

```

00001 #include "funkcijos.h"
00002
00003 #include <cassert>
00004 #include <iostream>
00005 #include <type_traits>
00006 #include <sstream>
00007
00008 VectorStudent tusciasStudentas() {
00009     return VectorStudent("", "", {}, 0, 0.0, 0.0);
00010 }
00011
00012 // Uzpildo studenta pastoviais duomenimis, kad testuose butu lengva lyginti rezultatus.
00013 void uzpildytiStudenta(VectorStudent& studentas) {
00014     studentas.setVardas("Jonas");
00015     studentas.setPavarde("Jonaitis");
00016     studentas.pazymiai().push_back(8);
00017     studentas.pazymiai().push_back(9);
00018     studentas.setEgzaminas(10);
00019     studentas.setVidurkis(9.2);
00020     studentas.setMediana(8.5);
00021 }
00022
00023 // Patikrina, ar studento duomenys sutampa su laukiamomis reikšnėmis.
00024 void patikrintiStudenta(const VectorStudent& studentas) {
00025     assert(studentas.vardas() == "Jonas");
00026     assert(studentas.pavarde() == "Jonaitis");
00027     assert(studentas.pazymiai().size() == 2);
00028     assert(studentas.pazymiai()[0] == 8);
00029     assert(studentas.pazymiai()[1] == 9);
00030     assert(studentas.egzaminas() == 10);
00031     assert(studentas.vidurkis() == 9.2);
00032     assert(studentas.mediana() == 8.5);
00033 }
00034
00035 void patikrintiTusciaStudenta(const VectorStudent& studentas) {
00036     assert(studentas.vardas().empty());
00037     assert(studentas.pavarde().empty());
00038     assert(studentas.pazymiai().empty());
00039     assert(studentas.egzaminas() == 0);
00040     assert(studentas.vidurkis() == 0.0);
00041     assert(studentas.mediana() == 0.0);
00042 }
00043
00044 // Tikrina konstruktoriu: ar sukurtas objektas turi tuscius/pradinius laukus.
00045 void test_empty_constructor_values() {
00046     static_assert(std::is_abstract_v<Zmogus>, "Zmogus klase privalo buti abstrakti.");
00047
00048     VectorStudent studentas = tusciasStudentas();
00049     patikrintiTusciaStudenta(studentas);
00050 }
00051
00052 // Tikrina paveldimuma ir tai, kad Studentas veikia per abstrakcios bazes sasaja.
00053 void test_inheritance_from_zmogus() {
00054     VectorStudent studentas = tusciasStudentas();
00055     studentas.setVardas("Jonas");
00056     studentas.setPavarde("Jonaitis");
00057
00058     Zmogus& zmogus = studentas;
00059
00060     assert(zmogus.vardas() == "Jonas");
00061     assert(zmogus.pavarde() == "Jonaitis");
00062     assert(zmogus.tipas() == "Studentas");
00063 }
00064

```

```

00065 // Tikrina kopijavimo konstruktoriu: ar sukuriamas pilna ir nepriklausoma kopija.
00066 void test_copy_constructor() {
00067     VectorStudent pirmas = tusciasStudentas();
00068     uzpildytiStudenta(pirmas);
00069
00070     VectorStudent antras(pirmas);
00071     patikrintiStudenta(antras);
00072
00073     pirmas.setVardas("Petras");
00074     pirmas.pazymiai().push_back(5);
00075
00076     assert(antras.vardas() == "Jonas");
00077     assert(antras.pazymiai().size() == 2);
00078 }
00079
00080 // Tikrina perkeliimo konstruktoriu: ar duomenys teisingai perkeliami i nauja objekta.
00081 void test_move_constructor() {
00082     VectorStudent pirmas = tusciasStudentas();
00083     uzpildytiStudenta(pirmas);
00084
00085     VectorStudent antras(std::move(pirmas));
00086     patikrintiStudenta(antras);
00087     patikrintiTusciaStudenta(pirmas);
00088 }
00089
00090 // Tikrina kopijavimo priskyrima: ar vienam objektui priskyrus kita, duomenys nukopijuojami teisingai.
00091 void test_copy_assignment() {
00092     VectorStudent pirmas = tusciasStudentas();
00093     VectorStudent antras = tusciasStudentas();
00094     uzpildytiStudenta(pirmas);
00095
00096     antras = pirmas;
00097     patikrintiStudenta(antras);
00098
00099     pirmas.setPavarde("Petraitis");
00100     pirmas.pazymiai().clear();
00101
00102     assert(antras.pavarde() == "Jonaitis");
00103     assert(antras.pazymiai().size() == 2);
00104 }
00105
00106 // Tikrina perkeliimo priskyrima: ar duomenys teisingai perkeliami jau egzistuojanciam objektui.
00107 void test_move_assignment() {
00108     VectorStudent pirmas = tusciasStudentas();
00109     VectorStudent antras = tusciasStudentas();
00110     uzpildytiStudenta(pirmas);
00111
00112     antras = std::move(pirmas);
00113     patikrintiStudenta(antras);
00114 }
00115
00116 // Tikrina destruktoriu: objektas sukuriamas ir sunaikinamas pasibaigus bloko sričiai.
00117 void test_destructor() {
00118     {
00119         VectorStudent studentas = tusciasStudentas();
00120         uzpildytiStudenta(studentas);
00121         assert(studentas.vardas() == "Jonas");
00122     }
00123 }
00124
00125 // Tikrina isvesties operatoriu « : ar studento duomenys isvedami laukiamu formatu.
00126 void test_output_operator() {
00127     VectorStudent studentas = tusciasStudentas();
00128     uzpildytiStudenta(studentas);
00129     std::ostringstream out;
00130
00131     out << studentas;
00132
00133     assert(out.str() == "Jonas Jonaitis 10 2 8 9");
00134 }
00135
00136 // Tikrina ivesties operatoriu » : ar duomenys teisingai nuskaitomi is srauto i objekta.
00137 void test_input_operator() {
00138     VectorStudent studentas = tusciasStudentas();
00139     std::istringstream in("Ona Onaite 9 3 10 8 7");
00140
00141     in >> studentas;
00142
00143     assert(studentas.vardas() == "Ona");
00144     assert(studentas.pavarde() == "Onaite");
00145     assert(studentas.egzaminas() == 9);
00146     assert(studentas.pazymiai().size() == 3);
00147     assert(studentas.pazymiai()[0] == 10);
00148     assert(studentas.pazymiai()[1] == 8);
00149     assert(studentas.pazymiai()[2] == 7);
00150 }
00151

```

```
00152 int main() {
00153     test_empty_constructor_values();
00154     test_inheritance_from_zmogus();
00155     test_copy_constructor();
00156     test_move_constructor();
00157     test_copy_assignment();
00158     test_move_assignment();
00159     test_destructor();
00160     test_output_operator();
00161     test_input_operator();
00162
00163     std::cout << "Visi rule of five ir operatoriu testai praejo." << std::endl;
00164     return 0;
00165 }
```

Index

- ~Studentas
 - Studentas< GradeContainer >, [12](#)
- ~Zmogus
 - Zmogus, [16](#)
- egzaminas
 - Studentas< GradeContainer >, [12](#)
- egzaminas_
 - Studentas< GradeContainer >, [14](#)
- mediana
 - Studentas< GradeContainer >, [12](#)
- mediana_
 - Studentas< GradeContainer >, [14](#)
- operator=
 - Studentas< GradeContainer >, [12](#)
 - Zmogus, [17](#)
- pavarde
 - Zmogus, [17](#)
- pavarde_
 - Zmogus, [18](#)
- pazymiai
 - Studentas< GradeContainer >, [13](#)
- pazymiai_
 - Studentas< GradeContainer >, [14](#)
- README, [1](#)
- setEgzaminas
 - Studentas< GradeContainer >, [13](#)
- setMediana
 - Studentas< GradeContainer >, [13](#)
- setPavarde
 - Zmogus, [17](#)
- setVardas
 - Zmogus, [17](#)
- setVidurkis
 - Studentas< GradeContainer >, [14](#)
- Studentas
 - Studentas< GradeContainer >, [11](#)
- Studentas< GradeContainer >, [9](#)
 - ~Studentas, [12](#)
 - egzaminas, [12](#)
 - egzaminas_, [14](#)
 - mediana, [12](#)
 - mediana_, [14](#)
 - operator=, [12](#)
 - pazymiai, [13](#)
 - pazymiai_, [14](#)

- setEgzaminas, [13](#)
- setMediana, [13](#)
- setVidurkis, [14](#)
- Studentas, [11](#)
- tipas, [14](#)
- vidurkis, [14](#)
- vidurkis_, [15](#)
- tipas
 - Studentas< GradeContainer >, [14](#)
 - Zmogus, [18](#)
- vardas
 - Zmogus, [18](#)
- vardas_
 - Zmogus, [18](#)
- vidurkis
 - Studentas< GradeContainer >, [14](#)
- vidurkis_
 - Studentas< GradeContainer >, [15](#)
- Zmogus, [15](#)
 - ~Zmogus, [16](#)
 - operator=, [17](#)
 - pavarde, [17](#)
 - pavarde_, [18](#)
 - setPavarde, [17](#)
 - setVardas, [17](#)
 - tipas, [18](#)
 - vardas, [18](#)
 - vardas_, [18](#)
 - Zmogus, [16](#)